

Stage 2A Report

Edge-Optimized Task-Driven Detection

A VEGA-FPGA Co-Design

Using Parallel Semantic and YOLO26n

Team Sitting Ducks

Bimsara Udurawana

Tharaka Jayasena

Thilakshan Balakrishnan

Department of Computer Science and Engineering
University of Moratuwa

Design Contest Stage 2A Report

Overview

Our model implements a prompt-driven object localization pipeline for edge deployment.

Input: natural-language prompt + image.

Output: bounding boxes around object(s) best matching the prompt's intent.

Architecture splits cleanly into a visual branch (YOLO detection) and a semantic branch (text embedding + cosine match against a fixed COCO vocabulary), enabling each branch to be partitioned independently between the host CPU and the FPGA fabric.

Why Model2Vec Over Other Encoders Evaluated

After benchmarked in against the same 80-class COCO matching task:

Encoder	Dims	GPU Latency (ms)	CPU Latency (ms)	R@1	R@5	R@10	MRR
MiniLM	384	0.218	1.096	0.660	0.926	0.956	0.778
MobileCLIP2-B	512	4.154	23.864	0.658	0.928	0.974	0.776
OpenCLIP-Distilled MobileCLIP2-B	256	3.091	22.925	0.608	0.847	0.895	0.713
Model2Vec	256	0.030	0.027	0.640	0.899	0.956	0.753

Fig 01: Encoder Performance

Metrics. Each prompt is ranked against the 80 COCO classes by cosine similarity vs the gold class set. $R@k$ = fraction of prompts with gold in top-k ($R@1$ = strict top-1). MRR = mean $1/\text{rank}$ of first gold hit. All in $[0, 1]$, higher better. $R@1$ and MRR matter most, fusion commits to the top-ranked class.

<https://github.com/BimsaraU/DVCon-SittingDucks>

No transformer ops: pure embedding lookup + averaging. No self-attention, no LayerNorm, no GELU. Eliminates the LUT/DSP-heavy blocks that dominate transformer FPGA accelerators.

Faster on CPU (0.027 ms vs 1.096ms): leaves the FPGA free for the visual branch and keeps the semantic path entirely host-side without becoming the bottleneck.

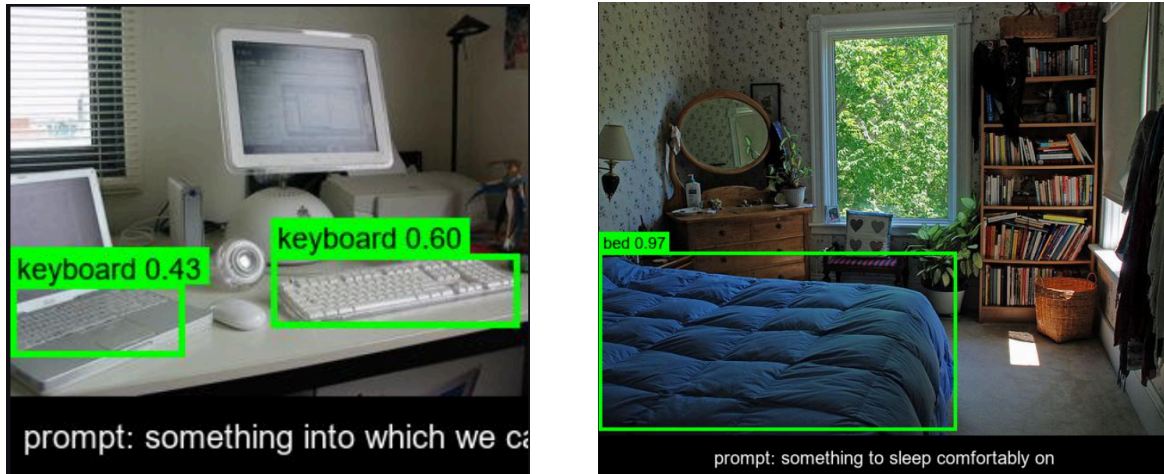
Tiny weight footprint: fits in host cache or, if offloaded, a small fraction of FPGA on-chip memory.

Quantization-friendly: static vectors quantize cleanly to INT8 with negligible ordering drift (verified by Spearman ρ and top-k Jaccard against MiniLM).

Negligible accuracy cost: $R@1$ drops only 2 points ($0.660 \rightarrow 0.640$) and $R@10$ is tied at 0.956; ranking quality is preserved where it matters for the fusion stage.

Deterministic, dependency-free runtime: no tokenizer/transformer library on the deployment image; simplifies the FPGA SoC's software stack.

Design Contest Stage 2A Report



```

prompt : 'something to sleep comfortably on'
image : D:\DVCon\DVCon-SittingDucks\images\000000000632.jpg
conf : 0.05
[yolo] loading D:\DVCon\DVCon-SittingDucks\Model\yolo26n.pt
[yolo] ready. classes=80
[m2v] loading from D:\DVCon\DVCon-SittingDucks\Model\potion-base-8M
[m2v] ready. coco_embs=(80, 256)
[parallel] detect=196.1ms semantic=196.1ms (wallclock=196.1ms)
[detect] raw detections: 34
[semantic] top5 classes for prompt:
bed 1.000
chair 0.651
clock 0.595
backpack 0.566
mouse 0.514
[rank] best class='bed' winner_count=1
bed score=0.975 sem=1.000 conf=0.899
best : bed score=0.975 sem=1.000 conf=0.899

prompt : 'something into which we can pour a drink'
image : D:\DVCon\DVCon-SittingDucks\images\00000001503.jpg
conf : 0.05
[yolo] loading D:\DVCon\DVCon-SittingDucks\Model\yolo26n.pt
[yolo] ready. classes=80
[m2v] loading from D:\DVCon\DVCon-SittingDucks\Model\potion-base-8M
[m2v] ready. coco_embs=(80, 256)
[parallel] detect=221.6ms semantic=221.6ms (wallclock=221.6ms)
[detect] raw detections: 5
[semantic] top5 classes for prompt:
wine glass 1.000
bottle 0.840
cup 0.740
spoon 0.720
fire hydrant 0.683
[rank] best class='keyboard' winner_count=2
keyboard score=0.602 sem=0.503 conf=0.898
keyboard score=0.432 sem=0.503 conf=0.220
best : keyboard score=0.602 sem=0.503 conf=0.898
saved : D:\DVCon\DVCon-SittingDucks\o1.jpg

```

Figure 03, 04 - Corresponding Terminal execution logs demonstrating the arbitration logic



Beyond the core functional requirements,

Our team explored two parallel optimization tracks to enhance performance and scalability:

[Hardware Acceleration](#): Investigated hardware-level acceleration to optimize inference speeds and reduce latency.

[Knowledge Graphs \(K-Graphs\)](#): Explored K-Graphs integration to improve intent resolution.